Gaussian Process Emulators

David Manning

July 7, 2020

1 Summary

Computer models of complex processes can take many hours of compute time, which makes people have to choose between waiting for the simulations to complete and buying large, powerful computers at high cost. In order to minimise the cost of each option, it is necessary to use alternative methods to infer what the computer model would produce without going to the expense of running it. One such method is to use a Gaussian process emulator, which finds a function which fits each of the points, and gives predictions for the other points based on the function. This method gives a vast improvement on other methods, such as linear interpolation, but are still computationally intensive on large and complex datasets, and should be used when estimating the output of computer code which model smooth processes.

2 Introduction

Computer models are often used to inform engineering projects, and these computer models are often based on very complicated and uninterpretable code which takes many hours to run for each set of inputs. Each extra variable that needs to be controlled for multiplies the total simulations required by the number of simulations taken for each variable, so the time taken to simulate data very quickly becomes too large to be able to run the simulations for as many data points as outputs are required (often in the thousands for each variable - of which there are likely to be many). This introduces the cost of buying, maintaining, and powering more or more powerful computers; waiting longer for results, potentially not having a solution until after it is too late; or a combination of the two.

Given these constraints, it is clearly necessary that a way to cut the cost of computing must be found. One possible suggestion is a linear interpolation method, however it has been suggested that a Gaussian process emulator, as proposed by O'Hagan (2006), may give better results. This report gives an analysis of Gaussian process emulators, and gives a comparison between its estimates and those from the linear interpolation method.

3 Gaussian Process Emulators

The Gaussian process emulator approach is used to estimate what output a deterministic computer model would give for input variables at different points without actually running the code for the points in question. This can be much faster than running the code an arbitrary number of times. There are two parts to the method, a simulator and

an emulator, which need to be run in the respective order.

The simulator runs the models for a variety of input values to give a set of known outputs in exactly the same way as the linear interpolation method.

Similar to the linear interpolation method, the emulator takes the input values and known output values and estimates and fits a function known as a "basis function" between the known points. As the points where the model was run are known, there is no estimation required there, and so the basis function will be constrained to give the required output at those values. The basis function can be any function, the simplest would be a straight line through both points (in which case its most likely value will be the same as that given by the linear interpolation method), however it could be something more complicated, to the point that it may take several minutes to compute for each input value. The basis function must be a unique solution to the points given, therefore, if there are only two points then the basis function must be a straight line. As more points are added, the basis function will gradually become closer and closer to the true function.

BACCO assumes that each function is smooth and continuous (and will not work when this is not true), so because we know the value of the output at the points fitted by the simulator, we know that the error in the estimate will be smaller when it is closer to a known point, and larger when it is further away from the known points. The emulator fits a standard error based on the assumption that the errors are normally distributed. The confidence intervals are intended to show the uncertainty in what the model would have predicted – it is not designed to add noise to the computer outputs.

Whereas the linear interpolation method can give predictions instantly, the Gaussian process emulator can need a lot of time to fit the model, and it does not scale well with larger datasets. The time complexity of an algorithm describes how much longer an algorithm will take to complete with higher sample sizes, usually defined as $O(f(n))$. Because it is designed to describe the complexity of running the algorithm on large datasets, if there are two or more parts which have a different complexity then the larger function will be used, and so $O(n^3+n^2)$ would become $O(n^3)$. The most complex part of fitting a Gaussian process is matrix multiplication, which runs in $O(n^3)$ time, and so these methods do not scale well to large datasets. However, the number of points needed to use linear interpolation rises exponentially; so whilst linear interpolation is quicker on a dataset after the models have been run, the Gaussian process emulator is more efficient when taking into account the time it takes to run the whole analysis.

4 Methods

The `GPfit` package (MacDonald et al. (2015)) in `R` (R Core Team (2017)) was used, with a tutorial on how to use the package given by the authors. A Gaussian Process emulator was fitted to the training data provided to provide a model. This model was then used to predict the model outputs from the test data, which gave MLEs and MSEs. The model accuracy was compared to both the linear interpolation method and the actual outputs.

The training set for the linear interpolation was for a different dataset with many more inputs, and unfortunately had several times more datasets with which to train its re-

sults, and so it would therefore be expected to give more accurate estimates as it has had more data to fit to, and smaller 'gaps' between the design points. For this reason, if the estimates provided by the Gaussian process emulation method are as accurate as those provided through linear interpolation, then the emulator should be considered to give better results than linear interpolation.

To compare the accuracy of the model outputs, three histograms of the model predictions were produced: one showing the difference between the most likely value of the emulator and the actual output. Because the Gaussian process emulator treats the errors as normally distributed, the median value should be around 0 if it is unbiased. In order to test whether the two estimators are unbiased and if the emulator gives estimates closer to the true value than interpolation, an exact binomial test was conducted. This test calculates the probability that we would see the number of over- or under- estimates for a specified probability and number of trials. The p-values this returns are treated in the same way as they are for any other statistical test, and should be corrected for the multiple comparison. A Šidak correction with a significance of α and m comparisons would require a significance level of $\alpha_3 = 1 - (1 - \alpha)^{1/m} = 1 - (1 - 0.05)^{1/3} = 0.017$. Observing a p-value of 0.017 with two comparisons represents the same interpretations as a p-value of 0.05 would normally.

5 Results

The emulator was closer to the real output in 61 of the test samples, and the binomial exact test gave a p-value of 0.01, which gives us some evidence that the emulator is more likely to give more accurate estimates than the linear interpolation method.

The histograms (shown in fig. 1 and fig. 2) comparing the estimates from the two methods to the actual computer output show that the estimate for the This is backed up by the results of the binomial exact tests: it gave a p-value of 0.028 for the emulator, providing no evidence that the emulator had biased estimates after taking a correction for multiple comparisons into account, however the binomial exact test for bias in the interpolation estimator gave a p-value of 8×10^{-9}, which represents overwhelming evidence that the interpolation estimate is systematically overestimating the output.

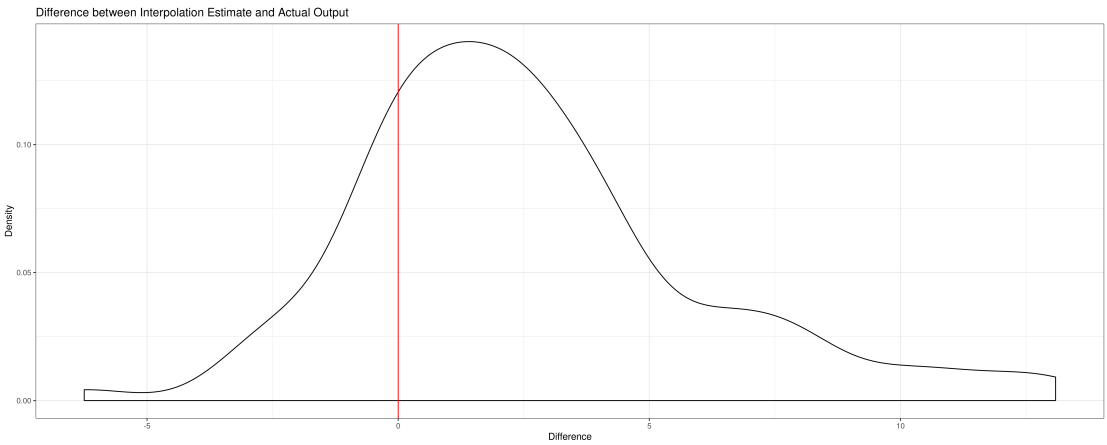

Figure 1: 77% of the estimates given by interpolation are above 0 – well outside the range expected if it were unbiased

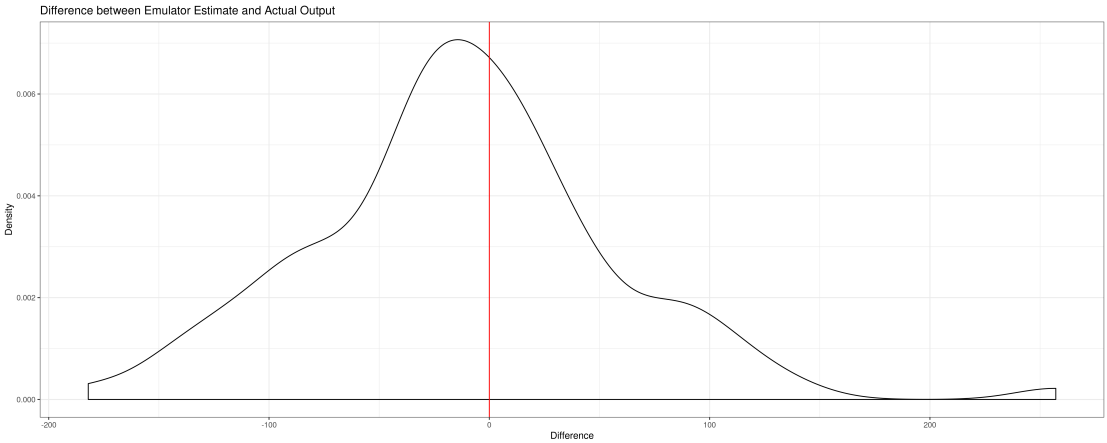

Figure 2: 40% of the estimates given by the emulator are above 0 – within the range expected if it were biased

With a required p-value of 0.025, this test would have shown no evidence against the null hypothesis if between 39 and 61 (inclusive) of the estimates were positive, and varying levels of evidence against if it were outside that range.

6 Conclusions and recommendations

Gaussian process emulators give much better answers than linear interpolation for several reasons. Firstly, because it is possible to see how wrong they are expected to be, it is possible to quantify the error and to know when there is uncertainty around the outputs. They also follow a function which is not necessarily linear, and which adjusts itself to the points it is being fit to, with smaller errors close to these points and as a result are unbiased.

These advantages also extend to time. Whereas linear interpolation is instantaneous, fitting models for Gaussian process emulation can be computationally intensive due to the matrix multiplication necessary to estimate the outputs, however they are not as intensive as producing the number of outputs necessary to be able to use linear interpolation. Once the models have been run, it is instantaneous to use the model to predict new output values in both methods.

Gaussian process emulation should certainly be used instead of linear interpolation when the models have not yet been run, and it should go without saying that if it is quicker to run the computer model for the desired outputs then that is preferable to using emulation.

There are also cases where Gaussian Process emulation is not appropriate. If the simulations can give differing outputs (for instance if they account for random variation in the output) then other methods should be used that accounts for the noise in the outputs. Even with deterministic models, BACCO does not add in the variation expected in the output, so if this is a requirement then the solution would be to use different modelling techniques – BACCO will only estimate the outputs that the simulator would have given, not what we may like it to have given. Gaussian processes only accurately

describe models where there is a smoothness in the data, where the underlying process is expected to suddenly change at certain points, other methods should be found in these cases.

Any further analysis comparing these two methods should, at a minimum, use the same training set for each method, and if possible several training sets with different sample sizes so the analysis can be redone with a fair test of each method, and also give the opportunity to test whether the computational expense of emulation is worth any benefits. This could also present an opportunity to test some other solutions to this problem.

References

MacDonald, B., Ranjan, P. and Chipman, H. (2015). Gpfit: An r package for fitting a gaussian process model to deterministic simulator outputs, *Journal of Statistical Software, Articles*, **64 (12)**: 1–23.

O'Hagan, A. (2006). Bayesian analysis of computer code outputs: A tutorial, *Reliability Engineering & System Safety*, **91 (10-11)**: 1290–1300.

R Core Team (2017). *R: A Language and Environment for Statistical Computing*, R Foundation for Statistical Computing, Vienna, Austria.

Rasmussen, C. E. and Williams, C. K. I. (2005). *Gaussian Processes for Machine Learning (Adaptive Computation and Machine Learning series)*, The MIT Press, ISBN 9780262182539.

Wickham, H. (2009). *ggplot2: Elegant Graphics for Data Analysis*, Springer-Verlag New York, ISBN 978-0-387-98140-6.

Wickham, H., François, R., Henry, L. and Müller, K. (2019). *dplyr: A Grammar of Data Manipulation*, R package version 0.8.3.